Help me understand ALL the concepts and code in the lab from its fundamentals. Explain as if to remove the necessity for further clarification or questioning, with layman's terms, logically structured and the assumption that the audience has low working memory capacity.

I'll go through the lab **concepts and code** step by step, in small chunks, with simple language. You can skim section titles and read them in order.

1. Big Picture: What This Lab Is Doing

The lab has **one small world** (a 5×5 grid) and several **learning methods**:

- First, you build a **gridworld environment**.
- Then you implement:
  - **ε-greedy** action selection.
  - **Monte Carlo (MC) prediction** – learn state values from full episodes.
  - **TD(0) prediction** – learn state values step-by-step.
  - **Q-learning and SARSA** – learn which actions to take (control).
  - (Optional) **Linear function approximation** – replace tables with a weight vector.

Think of it like this:

> The grid is a game board.
> The agent is a token on the board.
> Your algorithms are ways for the token to **learn good behaviour** (reach the goal, avoid traps, move efficiently).

2. The Gridworld Environment (Part 1)

2.1 What the Environment Represents

- The world is a **5×5 grid** (by default).
- Each **cell** is a position, like coordinates (x, y).
- There is:
  - A **start** cell (where the agent begins).
  - A **goal** cell (reward +1, episode ends).
  - One or more **traps** (negative reward when stepped into).
  - A **step penalty** (small negative reward per move) to encourage shorter paths.

So the agent wants to:

- Reach **G** (Goal) for +1.
- Avoid **T** (Traps) which are negative.
- Use **few steps** (because each step costs a little).

2.2 The Class Structure

You have:

```
class GridworldEnv:
    def __init__(self, width=5, height=5, start=(0,0), goal=(4,4),
                 traps=None, step_penalty=-0.01):
        self.width = width
        self.height = height
        self.start = start
        self.goal = goal
        self.traps = traps if traps is not None else {(2,2): -1.0}
        self.step_penalty = step_penalty
        self.n_actions = 4
        self.n_states = width * height
        self.reset()
```

Key ideas:

- **width, height**: grid size.
- **start, goal**: positions (x, y).
- **traps**: dictionary mapping positions to trap penalty, e.g. `(2,2): -1.0` means stepping into (2,2) gives −1.
- **step_penalty**: every move costs that amount (e.g. −0.01).
- **n_actions = 4**: Up, Right, Down, Left.
- **n_states = width * height**: each cell is a state; we numbering them 0...24.

You also have:

```
    def _to_state(self, pos):
        x, y = pos
        return y * self.width + x
```

- This turns (x, y) into a **single state index**.
  - e.g. in a 5-wide grid:
    - (0,0) → 0
    - (1,0) → 1
    - (0,1) → 5
  - It's row-major indexing: state = y * width + x.

```
    def reset(self):
        self.pos = self.start
        return self._to_state(self.pos)
```

- `reset()` puts the agent back at the start cell and returns its state index.

2.3 Implementing `step(self, action)`

The `step` function is the **core** of the environment:

> Input: an action (0=Up, 1=Right, 2=Down, 3=Left).
> Output: (next_state_index, reward, done_flag).

You need to:

1. Move the agent according to the action.
2. Handle boundaries (don't go outside the grid).
3. Compute reward:
   - Always add `step_penalty`.
   - Add goal reward (+1) if at goal.
   - Add trap penalty if at trap.
4. Set `done=True` if goal reached, else False.

Example implementation:

```
def step(self, action):
    x, y = self.pos

    # 1. Move according to action
    if action == 0:     # Up
        y -= 1
    elif action == 1:   # Right
        x += 1
    elif action == 2:   # Down
        y += 1
```

```
    elif action == 3:   # Left
        x -= 1

    # 2. Boundary handling: clamp x,y to [0, width-1] and [0, height-1]
    x = max(0, min(self.width - 1, x))
    y = max(0, min(self.height - 1, y))

    # Update position
    self.pos = (x, y)

    # 3. Reward computation
    reward = self.step_penalty  # base step cost

    done = False
    if self.pos == self.goal:
        reward += 1.0          # goal reward
        done = True
    elif self.pos in self.traps:
        reward += self.traps[self.pos]  # trap penalty

    # 4. Return next_state_index, reward, done
    next_state = self._to_state(self.pos)
    return next_state, reward, done
```

This is all you need for the environment to work.

3. ε-Greedy Action Selection and Helpers

3.1 ε-Greedy: Explore vs Exploit

You have a Q-table `Q` with shape `[n_states, n_actions]`. Row = state, column = action.

Goal of `epsilon_greedy(Q, state, epsilon, n_actions)`:

> With probability **ε**, choose a **random** action (exploration).
> With probability **1 – ε**, choose the **best** known action (exploitation).

Implementation:

```
def epsilon_greedy(Q, state, epsilon, n_actions):
    """Return an action using ε-greedy exploration."""
    if np.random.rand() < epsilon:
        # explore: random action
        return np.random.randint(n_actions)
    else:
        # exploit: greedy action
        return int(np.argmax(Q[state]))
```

- `np.random.rand()` returns a random number in [0,1).
- If it is less than ε → pick random action.
- Else → pick index of max Q[state].

3.2 `run_episode`

```
def run_episode(env, policy_fn, max_steps=200):
    """Generate an episode: list of (s, a, r)."""
    s = env.reset()
    traj = []
    for _ in range(max_steps):
        a = policy_fn(s)
        s2, r, done = env.step(a)
        traj.append((s, a, r))
        s = s2
        if done:
            break
    return traj
```

- `policy_fn` is a function: state → action.
- It repeatedly:
  - Chooses action `a`.
  - Steps environment → `s2, r, done`.
  - Stores (s, a, r).
  - Moves to `s2`.
  - Stops if `done` is True (goal reached).

`plot_learning_curve` just plots a list of returns vs episode.

4. Monte Carlo Prediction (Part 2)

Goal: estimate **V(s)**, the value of each state, using **first-visit Monte Carlo** under a **random policy**.

4.1 What Monte Carlo Does

For each episode:

1. Start at start state, follow a **random policy**.
2. Record the trajectory: `[(s0,a0,r1), (s1,a1,r2), ..., (sT,aT,r_{T+1})]`.
3. For each state's **first visit** in the episode:
   - Compute **return** G from that time step onward:
     $$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$
   - Add G to `returns_sum[state]`.
   - Increase `returns_count[state]` by 1.
4. Set `V[state] = returns_sum[state] / returns_count[state]`.

You do this for many episodes.

4.2 Code Skeleton

```
def mc_prediction(env, num_episodes=2000, gamma=0.99):
    V = np.zeros(env.n_states)
    returns_sum = np.zeros(env.n_states)
    returns_count = np.zeros(env.n_states)

    def random_policy(_s):
        return np.random.randint(env.n_actions)

    for _ in range(num_episodes):
        episode = run_episode(env, random_policy)
        # episode is list of (s, a, r)

        # 1. Extract states and rewards
        states = [step[^0] for step in episode]
        rewards = [step[^2] for step in episode]

        # 2. For each state, do first-visit MC
        visited = set()
        G = 0.0
        # Go backwards to accumulate returns
        for t in reversed(range(len(episode))):
            s_t = states[t]
            r_tp1 = rewards[t]  # reward after s_t

            G = gamma * G + r_tp1

            if s_t not in visited:
                visited.add(s_t)
                returns_sum[s_t] += G
                returns_count[s_t] += 1

    # 3. Compute V as averages
    nonzero = returns_count > 0
    V[nonzero] = returns_sum[nonzero] / returns_count[nonzero]
    return V
```

Key points:

- We walk **backwards** through the episode so G accumulates from the end.
- `visited` ensures **first-visit**: only the first time we see that state (from the start of episode) gets updated.

5. TD(0) Prediction (Part 3)

Goal: also estimate **V(s)**, but now using **TD(0)**, which updates **every step**, not at episode end.

5.1 TD(0) Idea

At each step:

1. You are in state s.
2. You take action under policy (here: random).
3. You see reward r and next state s2.
4. You update V(s) toward:
$$\text{target} = r + \gamma V(s2)$$
5. The update:
$$V(s) \leftarrow V(s) + \alpha\big(\text{target} - V(s)\big)$$

No need to wait for episode end.

5.2 Code

```
def td0_prediction(env, num_episodes=2000, alpha=0.1, gamma=0.99):
    V = np.zeros(env.n_states)

    def random_policy(_s):
        return np.random.randint(env.n_actions)

    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            a = random_policy(s)
            s2, r, done = env.step(a)

            # TD(0) update
            td_target = r + gamma * V[s2]
            td_error = td_target - V[s]
            V[s] += alpha * td_error

            s = s2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return V, returns
```

- `G_ep` is just the **total return of the episode** (for plotting).
- `V` is updated **online** step-by-step.

6. Control with Q-Learning and SARSA (Part 4)

Now we want not just **values of states**, but **which action to take** in each state. This means learning **Q(s,a)**.

6.1 Q-Learning (Off-Policy)

At each step:

1. From state `s`, choose action `a` using ε-greedy over Q.
2. Step environment → `s2, r, done`.
3. Compute:
$$\text{target} = r + \gamma \max_{a'} Q(s2, a')$$
4. Update:
$$Q(s,a) \leftarrow Q(s,a) + \alpha(\text{target} - Q(s,a))$$

Note: we use the **greedy** `max` in the target, even if our actual behavior is ε-greedy. That's why it's **off-policy**.

Code:

```
def q_learning(env, num_episodes=3000, alpha=0.1, gamma=0.99, epsilon=0.1):
    Q = np.zeros((env.n_states, env.n_actions))
    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            a = epsilon_greedy(Q, s, epsilon, env.n_actions)
            s2, r, done = env.step(a)

            # Q-learning update
            best_next = np.max(Q[s2])           # max over actions at s2
            td_target = r + gamma * best_next
            td_error = td_target - Q[s, a]
            Q[s, a] += alpha * td_error

            s = s2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return Q, returns
```

6.2 SARSA (On-Policy)

SARSA update uses the **actual next action** chosen, not the max.

At each step:

1. From state `s`, choose `a` with ε-greedy.

2. Step env → `s2, r, done`.
3. Choose **next action** `a2` from `s2` with ε-greedy.
4. Compute:
$$\text{target} = r + \gamma Q(s2, a2)$$
5. Update:
$$Q(s,a) \leftarrow Q(s,a) + \alpha(\text{target} - Q(s,a))$$

Code:

```
def sarsa(env, num_episodes=3000, alpha=0.1, gamma=0.99, epsilon=0.1):
    Q = np.zeros((env.n_states, env.n_actions))
    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        a = epsilon_greedy(Q, s, epsilon, env.n_actions)
        G_ep = 0.0
        for _ in range(200):
            s2, r, done = env.step(a)
            a2 = epsilon_greedy(Q, s2, epsilon, env.n_actions)

            # SARSA update
            td_target = r + gamma * Q[s2, a2]
            td_error = td_target - Q[s, a]
            Q[s, a] += alpha * td_error

            s, a = s2, a2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return Q, returns
```

Difference in words:

- **Q-learning** assumes next action will be **greedy** (best).
- **SARSA** assumes next action will be **whatever the ε-greedy policy actually does**, including exploration.

So:

- Q-learning = more **optimistic/aggressive**.
- SARSA = more **conservative/safe**.

6.3 Visualising the Greedy Policy

You already have:

```
ARROWS = {0:'↑', 1:'→', 2:'↓', 3:'←'}

def show_greedy_policy(env, Q):
```

```
    grid = np.full((env.height, env.width), '·', dtype=object)
    for y in range(env.height):
        for x in range(env.width):
            if (x,y) == env.goal:
                grid[y,x] = 'G'
            elif (x,y) in env.traps:
                grid[y,x] = 'T'
            else:
                s = y * env.width + x
                grid[y,x] = ARROWS[int(np.argmax(Q[s]))]
    print('\n'.join(' '.join(row) for row in grid))
```

- For each cell (x,y), it:
  - If it's goal → show 'G'.
  - If it's trap → show 'T'.
  - Else → compute state index s, pick greedy action `argmax(Q[s])`, draw arrow.

This lets you **see** the learned behaviour.

7. Optional: Linear Function Approximation for Q(s,a)

Now we replace the table Q[s, a] by a **feature vector** and a **weight vector**.

7.1 Feature Mapping `phi(env, s, a)`

Goal: map (state, action) → feature vector φ(s,a).

We're asked to:

- Create a **one-hot** vector for state (length = n_states).
- Create a **one-hot** vector for action (length = n_actions).
- Concatenate them.

Example:
Suppose n_states=25, n_actions=4.
For state s=3, action a=2:

- state one-hot = [0,0,0,1,0,0,...,0] (25 long).
- action one-hot = (4 long).[1]
- φ(s,a) = [state_one_hot, action_one_hot] → length 29.

Code:

```
def phi(env, s, a):
    # state part
    state_feat = np.zeros(env.n_states)
    state_feat[s] = 1.0

    # action part
```

```
    action_feat = np.zeros(env.n_actions)
    action_feat[a] = 1.0

    # concatenate
    return np.concatenate([state_feat, action_feat])
```

7.2 Semi-Gradient Linear Q-Learning

We maintain a weight vector `w` of length `d = env.n_states + env.n_actions`.

At each step:

1. For current (s, a), compute prediction:
$$\hat{Q}(s, a) = \phi(s, a)^T w$$
2. Compute target:
$$\text{target} = r + \gamma \max_{a'} \hat{Q}(s2, a')$$
   where:
$$\hat{Q}(s2, a') = \phi(s2, a')^T w$$
3. Error:
$$\delta = \text{target} - \hat{Q}(s, a)$$
4. Update weights:
$$w \leftarrow w + \alpha \, \delta \, \phi(s, a)$$

Code:

```
def q_linear(env, num_episodes=5000, alpha=0.05, gamma=0.99,
             epsilon=0.1):
    d = env.n_states + env.n_actions
    w = np.zeros(d)
    returns = []
    for _ in range(num_episodes):
        s = env.reset()
        G_ep = 0.0
        for _ in range(200):
            # compute Q values for all actions at current s
            q_vals = np.array([phi(env, s, a) @ w
                               for a in range(env.n_actions)])
            # epsilon-greedy over these q_vals
            if np.random.rand() < epsilon:
                a = np.random.randint(env.n_actions)
            else:
                a = int(np.argmax(q_vals))

            s2, r, done = env.step(a)

            # compute Q values at next state
            q_vals_next = np.array([phi(env, s2, a2) @ w
                                    for a2 in range(env.n_actions)])
            best_next = np.max(q_vals_next)
```

```
            # target and prediction
            pred = phi(env, s, a) @ w
            target = r + gamma * best_next
            td_error = target - pred

            # semi-gradient update
            w += alpha * td_error * phi(env, s, a)

            s = s2
            G_ep += r
            if done:
                break
        returns.append(G_ep)
    return w, returns
```

This is the same logic as tabular Q-learning, but Q is computed by `w` and `phi` instead of a table.

8. Concept Wrap-Up

To summarise the key ideas briefly:

- **Environment (`GridworldEnv`):** small MDP where the agent moves on a grid, with start, goal, traps, and step penalty.
- **ε-greedy**: with probability ε choose random action; else choose action with highest Q.
- **Monte Carlo prediction**: learn V(s) using full episode returns, first-visit rule.
- **TD(0) prediction**: learn V(s) step-by-step using $r + \gamma V(s')$.
- **Q-learning**: learn Q(s,a), off-policy, target uses `max_a' Q(s', a')`.
- **SARSA**: learn Q(s,a), on-policy, target uses `Q(s', a_next)` where `a_next` is ε-greedy.
- **Linear function approximation**: replace Q table by $Q(s,a) \approx \varphi(s,a)^T w$ and update `w` with semi-gradient.

If you paste in your current **incomplete code**, I can fill in the missing TODOs exactly and annotate them line-by-line.

⁂

1. COMP40771_L5_Reinforcement_Learning.pdf